JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Class ServerSocketChannel

java.lang.Object
  java.nio.channels.spi.AbstractInterruptibleChannel
    java.nio.channels.SelectableChannel
      java.nio.channels.spi.AbstractSelectableChannel
        java.nio.channels.ServerSocketChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `Channel`, `InterruptibleChannel`, `NetworkChannel`

```
public abstract class ServerSocketChannel
extends AbstractSelectableChannel
implements NetworkChannel
```

A selectable channel for stream-oriented listening sockets.

A server-socket channel is created by invoking the `open` method of this class. It is not possible to create a channel for an arbitrary, pre-existing `ServerSocket`. A newly-created server-socket channel is open but not yet bound. An attempt to invoke the `accept` method of an unbound server-socket channel will cause a `NotYetBoundException` to be thrown. A server-socket channel can be bound by invoking one of the `bind` methods defined by this class.

Socket options are configured using the `setOption` method. Server-socket channels support the following options:

Option Name	Description
`SO_RCVBUF`	The size of the socket receive buffer
`SO_REUSEADDR`	Re-use address

Additional (implementation specific) options may also be supported.

Server-socket channels are safe for use by multiple concurrent threads.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`ServerSocketChannel(SelectorProvider provider)`	Initializes a new instance of this class.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
abstract `SocketChannel`	`accept()`	Accepts a connection made to this channel's socket.
`ServerSocketChannel`	`bind(SocketAddress local)`	Binds the channel's socket to a local address and configures the socket to listen for connections.

abstract **ServerSocketChannel**	**bind**(**SocketAddress** local, int backlog)	Binds the channel's socket to a local address and configures the socket to listen for connections.
abstract **SocketAddress**	**getLocalAddress**()	Returns the socket address that this channel's socket is bound to.
static **ServerSocketChannel**	**open**()	Opens a server-socket channel.
abstract <T> **ServerSocketChannel**	**setOption**(**SocketOption**<T> name, T value)	Sets the value of a socket option.
abstract **ServerSocket**	**socket**()	Retrieves a server socket associated with this channel.
int	**validOps**()	Returns an operation set identifying this channel's supported operations.

Methods inherited from class java.nio.channels.spi.AbstractSelectableChannel

blockingLock, configureBlocking, implCloseChannel, implCloseSelectableChannel, implConfigureBlocking, isBlocking, isRegistered, keyFor, provider, register

Methods inherited from class java.nio.channels.SelectableChannel

register

Methods inherited from class java.nio.channels.spi.AbstractInterruptibleChannel

begin, close, end, isOpen

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods inherited from interface java.nio.channels.NetworkChannel

getOption, supportedOptions

Methods inherited from interface java.nio.channels.Channel

close, isOpen

Constructor Detail

ServerSocketChannel

```
protected ServerSocketChannel(SelectorProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider - The provider that created this channel`

Method Detail

open

```
public static ServerSocketChannel open()
                                 throws IOException
```

Opens a server-socket channel.

The new channel is created by invoking the `openServerSocketChannel` method of the system-wide default `SelectorProvider` object.

The new channel's socket is initially unbound; it must be bound to a specific address via one of its socket's `bind` methods before connections can be accepted.

Returns:

A new socket channel

Throws:

`IOException` - If an I/O error occurs

validOps

```
public final int validOps()
```

Returns an operation set identifying this channel's supported operations.

Server-socket channels only support the accepting of new connections, so this method returns `SelectionKey.OP_ACCEPT`.

Specified by:

`validOps` in class `SelectableChannel`

Returns:

The valid-operation set

bind

```
public final ServerSocketChannel bind(SocketAddress local)
                                throws IOException
```

Binds the channel's socket to a local address and configures the socket to listen for connections.

An invocation of this method is equivalent to the following:

```
bind(local, 0);
```

Specified by:

`bind` in interface `NetworkChannel`

Parameters:

`local` - The local address to bind the socket, or null to bind to an automatically assigned socket address

Returns:

This channel

Throws:

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If the channel is closed

`IOException` - If some other I/O error occurs

`SecurityException` - If a security manager has been installed and its `checkListen` method denies the operation

Since:

1.7

See Also:

`NetworkChannel.getLocalAddress()`

bind

```
public abstract ServerSocketChannel bind(SocketAddress local,
                                         int backlog)
                                  throws IOException
```

Binds the channel's socket to a local address and configures the socket to listen for connections.

This method is used to establish an association between the socket and a local address. Once an association is established then the socket remains bound until the channel is closed.

The `backlog` parameter is the maximum number of pending connections on the socket. Its exact semantics are implementation specific. In particular, an implementation may impose a maximum length or may choose to ignore the parameter altogther. If the `backlog` parameter has the value `0`, or a negative value, then an implementation specific default is used.

Parameters:

`local` - The address to bind the socket, or null to bind to an automatically assigned socket address

`backlog` - The maximum number of pending connections

Returns:

This channel

Throws:

AlreadyBoundException - If the socket is already bound

UnsupportedAddressTypeException - If the type of the given address is not supported

ClosedChannelException - If this channel is closed

IOException - If some other I/O error occurs

SecurityException - If a security manager has been installed and its checkListen method denies the operation

Since:

1.7

setOption

```
public abstract <T> ServerSocketChannel setOption(SocketOption<T> name,
                                                  T value)
                                           throws IOException
```

Description copied from interface: NetworkChannel

Sets the value of a socket option.

Specified by:

setOption in interface NetworkChannel

Type Parameters:

T - The type of the socket option value

Parameters:

name - The socket option

value - The value of the socket option. A value of null may be a valid value for some socket options.

Returns:

This channel

Throws:

UnsupportedOperationException - If the socket option is not supported by this channel

IllegalArgumentException - If the value is not a valid value for this socket option

ClosedChannelException - If this channel is closed

IOException - If an I/O error occurs

Since:

1.7

See Also:

StandardSocketOptions

socket

```
public abstract ServerSocket socket()
```

Retrieves a server socket associated with this channel.

The returned object will not declare any public methods that are not declared in the ServerSocket class.

Returns:

A server socket associated with this channel

accept

```
public abstract SocketChannel accept()
                               throws IOException
```

Accepts a connection made to this channel's socket.

If this channel is in non-blocking mode then this method will immediately return `null` if there are no pending connections. Otherwise it will block indefinitely until a new connection is available or an I/O error occurs.

The socket channel returned by this method, if any, will be in blocking mode regardless of the blocking mode of this channel.

This method performs exactly the same security checks as the accept method of the ServerSocket class. That is, if a security manager has been installed then for each new connection this method verifies that the address and port number of the connection's remote endpoint are permitted by the security manager's checkAccept method.

Returns:

The socket channel for the new connection, or null if this channel is in non-blocking mode and no connection is available to be accepted

Throws:

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the accept operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the accept operation is in progress, thereby

closing the channel and setting the current thread's interrupt status

NotYetBoundException - If this channel's socket has not yet been bound

SecurityException - If a security manager has been installed and it does not permit access to the remote endpoint of the new connection

IOException - If some other I/O error occurs

getLocalAddress

```
public abstract SocketAddress getLocalAddress()
                                        throws IOException
```

Returns the socket address that this channel's socket is bound to.

Where the channel is bound to an Internet Protocol socket address then the return value from this method is of type InetSocketAddress.

If there is a security manager set, its checkConnect method is called with the local address and -1 as its arguments to see if the operation is allowed. If the operation is not allowed, a SocketAddress representing the loopback address and the local port of the channel's socket is returned.

Specified by:

getLocalAddress in interface NetworkChannel

Returns:

The SocketAddress that the socket is bound to, or the SocketAddress representing the loopback address if denied by the security manager, or null if the channel's socket is not bound

Throws:

ClosedChannelException - If the channel is closed

```
IOException - If an I/O error occurs
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.